

DreamSupport

A production-ready multilingual Voice AI customer support agent with continuous self-improvement.

"We didn't just build a Voice AI Agent. We built an AI Agent that becomes better every day."

Table of Contents

1. [Overview](#)
 2. [Vision](#)
 3. [Problem Statement](#)
 4. [Objectives](#)
 5. [Technology Stack](#)
 6. [System Architecture](#)
 7. [Ultra-Low-Latency Voice Pipeline](#)
 8. [Multilingual Pipeline](#)
 9. [Memory Architecture](#)
 10. [Retrieval Pipeline](#)
 11. [LangGraph Workflow](#)
 12. [The Self-Improvement Engine](#)
 13. [The Dream Cycle](#)
 14. [Dream Cycle Interruptions](#)
 15. [What DreamSupport Learns](#)
 16. [Execution Traces](#)
 17. [Observability](#)
 18. [Evaluation & Metrics](#)
 19. [Why DreamSupport Is Different](#)
 20. [Future Evolution: DreamForge](#)
 21. [Glossary](#)
-

1. Overview

DreamSupport is a production-grade, multilingual Voice AI customer support agent capable of holding natural, real-time conversations with customers — while continuously

improving itself through an autonomous process called the **Dream Cycle**.

Unlike traditional AI support systems that stop learning the moment they are deployed, DreamSupport analyzes its own conversations, discovers its mistakes, evaluates alternative reasoning paths, optimizes retrieval, consolidates memory, and evolves its decision-making over time.

The agent literally "**dreams**" whenever it has idle time:

- **No model retraining.**
 - **No human supervision.**
 - **Continuous improvement from experience.**
-

2. Vision

DreamSupport is built on a simple but ambitious premise: an AI agent should not be frozen at the moment of deployment. It should get smarter the longer it operates.

Where conventional systems store conversations and never learn from them, DreamSupport treats **every conversation as training material**. During idle time it replays, critiques, and refines its own behavior — improving the intelligence *around* the language model rather than the model's weights themselves.

The result is an agent that serves each customer slightly better than the last.

3. Problem Statement

Today's AI customer support systems share a common set of limitations:

- High response latency.
- Usually support only one language well.
- Cannot continuously improve after deployment.
- Repeat identical mistakes.
- Store conversations but never learn from them.
- Memory grows without organization.
- Retrieval quality degrades over time.
- Difficult to debug *why* the AI answered something.
- Developers cannot easily measure improvement.

DreamSupport solves all of these problems within a single production-ready architecture.

4. Objectives

DreamSupport is designed to:

Goal	Description
Real-time voice	Support natural, low-latency spoken conversations.
Multilingual	Support multiple Indian languages.
Language detection	Detect the spoken language automatically.
Language preservation	Keep the conversation in the same language end-to-end.
Knowledge grounding	Use RAG over enterprise knowledge.
Streaming	Stream responses with very low latency.
Traceability	Store complete execution traces.
Self-improvement	Continuously improve without retraining.
Better retrieval	Learn improved retrieval strategies.
Better planning	Learn improved planning strategies.
Memory hygiene	Organize and consolidate long-term memory.
Observability	Provide complete visibility into every decision.

5. Technology Stack

Layer	Technology
Backend	Python, FastAPI
Voice Pipeline	Pipecat
Voice Activity Detection	Silero VAD
Speech-to-Text	Sarvam STT

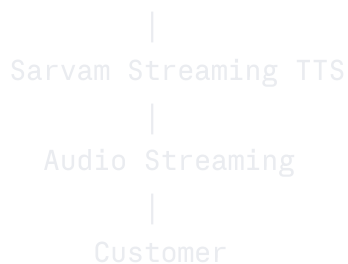
Layer	Technology
Text-to-Speech	Sarvam TTS
Agent Framework	LangGraph
Observability	LangSmith
Vector Database	Qdrant
Relational Database	PostgreSQL
Caching	Redis
Object Storage	S3-compatible storage
Embeddings	Configurable
LLM	Configurable

The **embeddings model** and **LLM** are intentionally configurable so the system can be tuned for cost, latency, language coverage, or on-premise deployment.

6. System Architecture

The end-to-end request flow, from the customer's voice to a spoken reply:





Each stage streams into the next, so the customer hears the beginning of an answer before the full answer has finished generating.

7. Ultra-Low-Latency Voice Pipeline

Traditional voice pipelines are sequential and blocking — each stage must fully complete before the next begins:



DreamSupport instead streams every stage so work overlaps:

```
DreamSupport Pipeline
  User Speaks
    ↓
  Silero detects speech
    ↓
  Streaming STT begins
    ↓
  LLM starts immediately
    ↓
  Tokens streamed instantly
    ↓
  Streaming TTS begins
    ↓
  Customer hears first words while the
  remaining response is still being generated.
```

This overlapping design is what gives DreamSupport its perceived real-time responsiveness.

8. Multilingual Pipeline

A defining design rule: **the conversation never changes language internally**. There is no intermediate translation to English.

Example

```
Customer: "माझं ऑर्डर अजून आलं नाही."
  ↓ Sarvam STT
Marathi Text
  ↓ Retriever
  ↓ LLM
Marathi Text
  ↓ Sarvam TTS
Marathi Audio
```

Advantages of staying in the original language:

- Lower latency (no translation hops).
- Better meaning preservation.
- Better multilingual reasoning.
- More natural responses.

9. Memory Architecture

DreamSupport maintains three distinct memory layers:

1. Conversation Memory

The live context of the current call: the current issue, recent turns, and immediate context.

2. Customer Memory

Who this person is across time: previous conversations, customer profile, preferences, and past issues.

3. Knowledge Memory

What the company knows: documentation, FAQs, policies, the knowledge base, and product manuals.

Together these layers let the agent be both *contextually aware* (this conversation) and *grounded* (company truth).

10. Retrieval Pipeline

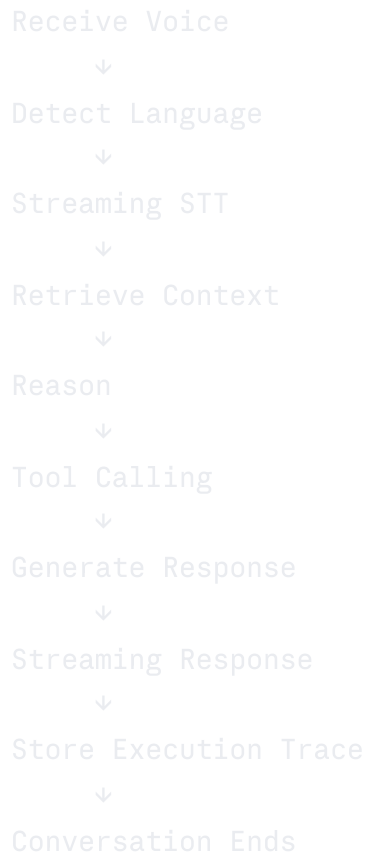
Retrieval-Augmented Generation grounds every answer in enterprise knowledge:



The reranker is optional and can be enabled when retrieval precision matters more than latency. Crucially, this pipeline is one of the components the Dream Cycle learns to improve over time.

11. LangGraph Workflow

LangGraph orchestrates the full per-conversation workflow:



The final step — **storing the execution trace** — is what makes self-improvement possible. Nothing is improved that wasn't first recorded.

12. The Self-Improvement Engine

This is DreamSupport's most unique capability.

Instead of stopping after every conversation, the agent **continuously learns from itself**. Every conversation becomes training material.

The Dream Engine is built **inside** DreamSupport. No external framework is required.

13. The Dream Cycle

Whenever there are no active customer conversations, DreamSupport enters **Dream Mode** and runs the following cycle:

```
graph TD; A[Conversation Ends] --> B[No Active Customers]; B --> C[Dream Cycle Starts]; C --> D[Replay Previous Conversations]; D --> E[Analyze Failures]; E --> F[Evaluate Retrieval Quality]; F --> G[Analyze Tool Usage]; G --> H[Discover Better Planning]; H --> I[Generate Alternative Solutions]; I --> J[Compare Different Reasoning Paths]; J --> K[Generate Synthetic Customer Queries]; K --> L[Test New Strategies]; L --> M[Discover Better Policies]; M --> N[Consolidate Memory]; N --> O[Remove Duplicate Memories]; O --> P[Strengthen Useful Memories]; P --> Q[Generate Candidate Improvements]; Q --> R[Checkpoint Saved]; R --> S[Sleep];
```

Conversation Ends

↓

No Active Customers

↓

Dream Cycle Starts

↓

Replay Previous Conversations

↓

Analyze Failures

↓

Evaluate Retrieval Quality

↓

Analyze Tool Usage

↓

Discover Better Planning

↓

Generate Alternative Solutions

↓

Compare Different Reasoning Paths

↓

Generate Synthetic Customer Queries

↓

Test New Strategies

↓

Discover Better Policies

↓

Consolidate Memory

↓

Remove Duplicate Memories

↓

Strengthen Useful Memories

↓

Generate Candidate Improvements

↓

Checkpoint Saved

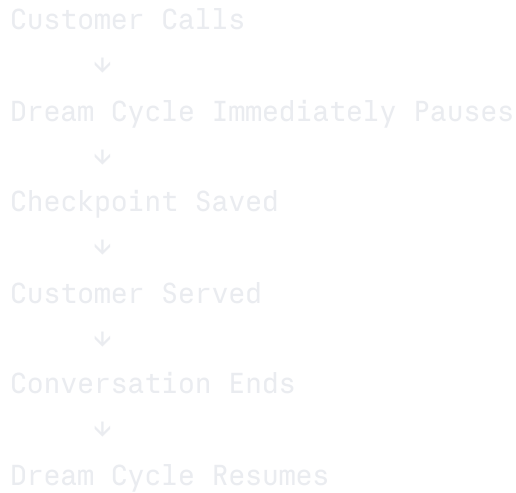
↓

Sleep

The cycle is a loop: it runs, checkpoints its progress, sleeps, and resumes whenever idle time is available again.

14. Dream Cycle Interruptions

The customer **always** has priority. The Dream Engine behaves like a background process that yields instantly:



Because progress is checkpointed before pausing, no dreaming work is lost when a customer interrupts.

15. What DreamSupport Learns

DreamSupport **does not retrain the LLM**. Instead it improves everything around it — the *intelligence surrounding* the model:

- Better retrieval order
- Better planning
- Better prompts
- Better tool selection
- Better memory organization
- Better document selection
- Better response templates
- Better conversation strategies
- Better reasoning workflows

The LLM stays fixed; the system around it evolves.

Every conversation stores a complete trace — everything required for dreaming:

- User input
- Detected language
- Retrieved documents
- Tool calls
- Planning graph
- Memory access
- Reasoning metadata
- Response
- Latency
- Evaluation
- Customer feedback

These traces are the raw material the Dream Cycle replays and critiques.

17. Observability

LangSmith provides full visibility into the system:

- Complete execution graph
- Token usage
- Latency
- Prompt traces
- Retrieval traces
- Tool traces
- Evaluation metrics
- Failure debugging
- Dream Cycle logs

This makes it possible to answer the question that most AI support systems cannot: *why did the AI answer the way it did?*

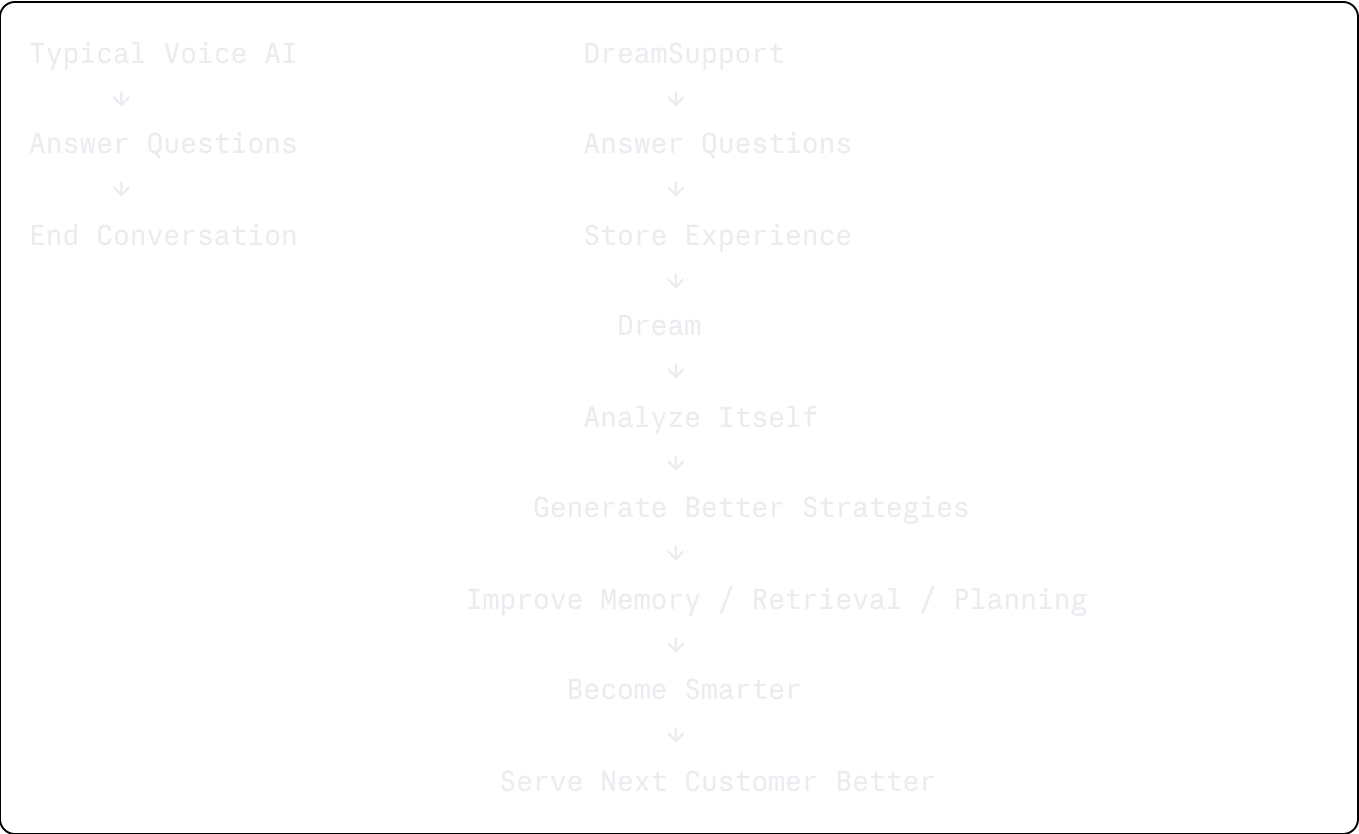
18. Evaluation & Metrics

DreamSupport continuously measures both live performance and self-improvement:

Category	Metrics
Latency	Response latency, streaming latency
Retrieval	Retrieval precision, groundedness, hallucination rate
Task quality	Task completion, customer satisfaction
Language	Language consistency
Memory	Memory growth, memory compression
Self-improvement	Dream improvement score, strategy improvement

Tracking improvement metrics over time is how developers verify that dreaming is actually making the agent better.

19. Why DreamSupport Is Different

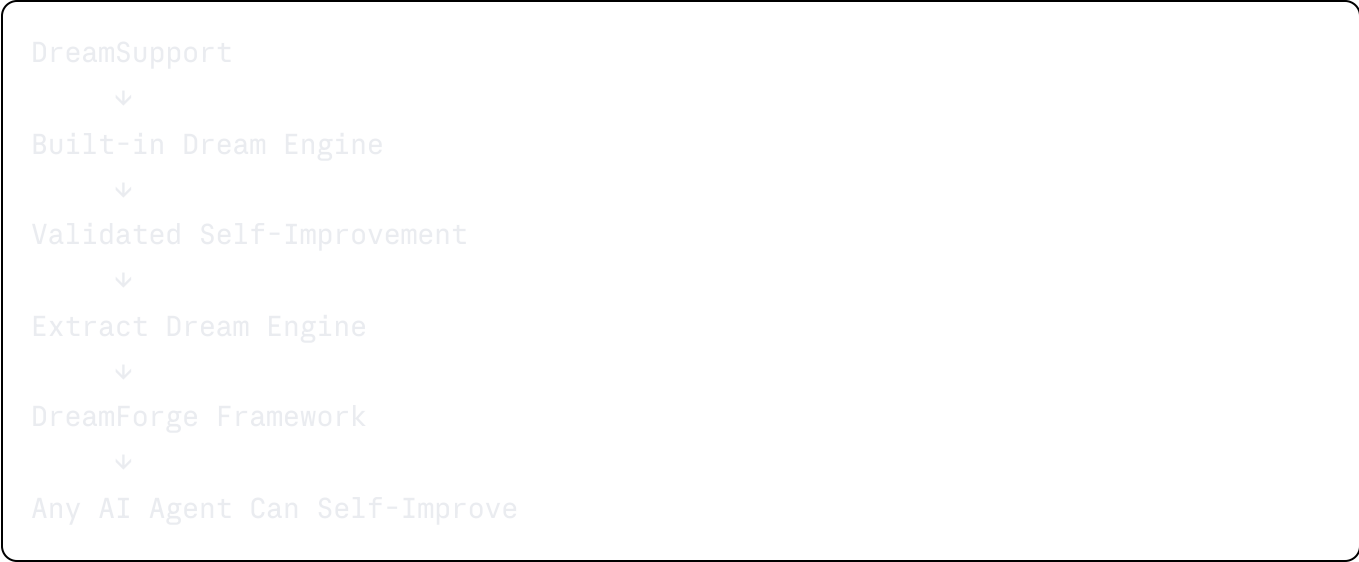


A typical agent ends where DreamSupport begins its second life.

20. Future Evolution: DreamForge

The Dream Engine inside DreamSupport will eventually be extracted into a standalone, reusable framework called **DreamForge**. DreamSupport will then become the first production application powered by DreamForge.

Development Roadmap



The end goal: a framework that lets *any* AI agent self-improve from its own experience.

21. Glossary

Term	Meaning
Dream Cycle	The autonomous, idle-time process in which the agent replays and improves from its own conversations.
Dream Mode	The state the agent enters when no customers are active and the Dream Cycle runs.
Execution Trace	The complete recorded record of a single conversation, used as training material for dreaming.
DreamForge	The future standalone framework extracted from DreamSupport's Dream Engine.
RAG	Retrieval-Augmented Generation — grounding LLM answers in retrieved enterprise knowledge.
VAD	Voice Activity Detection — identifying when speech is present in audio.

Term	Meaning
Checkpoint	A saved snapshot of Dream Cycle progress, allowing safe pause and resume.

DreamSupport — an AI agent that becomes better every day.